

DAA-All Algorithms

Unit-2 Algorithms (7)

Divide And Conquer (3):

1. Binary Search
2. Quick Sort
3. Merge Sort

Disjoint Sets (4):

4. Simple Union
5. Simple Find
6. Weighted Union
7. Collapsing Find

1. Binary Search : Iterative Algorithm

Algorithm BinSearch(a, n, x)
// Given an array $a[1 : n]$ of elements in nondecreasing
// order, $n \geq 0$, determine whether x is present, and
// if so, return j such that $x = a[j]$; else return 0.
{
 $low := 1; high := n;$
 while ($low \leq high$) **do**
 {
 $mid := \lfloor (low + high)/2 \rfloor;$
 if ($x < a[mid]$) **then** $high := mid - 1;$
 else if ($x > a[mid]$) **then** $low := mid + 1;$
 else return $mid;$
 }
 return 0;
}

1. Binary Search : Recursive Algorithm

Algorithm BinSrch(a, i, l, x)
// Given an array $a[i : l]$ of elements in nondecreasing
// order, $1 \leq i \leq l$, determine whether x is present, and
// if so, return j such that $x = a[j]$; else return 0.
{
 if ($l = i$) **then** // If Small(P)
 {
 if ($x = a[i]$) **then return** i ;
 else return 0;
 }
 else
 { // Reduce P into a smaller subproblem.
 $mid := \lfloor (i + l) / 2 \rfloor$;
 if ($x = a[mid]$) **then return** mid ;
 else if ($x < a[mid]$) **then**
 return BinSrch($a, i, mid - 1, x$);
 else return BinSrch($a, mid + 1, l, x$);
 }
}

2. Quick Sort

```
Algorithm QuickSort( $p, q$ )  
// Sorts the elements  $a[p], \dots, a[q]$  which reside in the global  
// array  $a[1 : n]$  into ascending order;  $a[n + 1]$  is considered to  
// be defined and must be  $\geq$  all the elements in  $a[1 : n]$ .  
{  
    if ( $p < q$ ) then // If there are more than one element  
    {  
        // divide  $P$  into two subproblems.  
         $j := \text{Partition}(a, p, q + 1);$   
        //  $j$  is the position of the partitioning element.  
        // Solve the subproblems.  
        QuickSort( $p, j - 1$ );  
        QuickSort( $j + 1, q$ );  
        // There is no need for combining solutions.  
    }  
}
```

2. Quick Sort

Algorithm Partition(a, m, p)

// Within $a[m], a[m+1], \dots, a[p-1]$ the elements are
// rearranged in such a manner that if initially $t = a[m]$,
// then after completion $a[q] = t$ for some q between m
// and $p-1$, $a[k] \leq t$ for $m \leq k < q$, and $a[k] \geq t$
// for $q < k < p$. q is returned. Set $a[p] = \infty$.

```
{
     $v := a[m]; i := m; j := p;$ 
    repeat
    {
        repeat
             $i := i + 1;$ 
        until  $(a[i] \geq v);$ 

        repeat
             $j := j - 1;$ 
        until  $(a[j] \leq v);$ 

        if  $(i < j)$  then Interchange( $a, i, j$ );
    } until  $(i \geq j);$ 
     $a[m] := a[j]; a[j] := v;$  return  $j;$ 
}
```

Algorithm Interchange(a, i, j)

// Exchange $a[i]$ with $a[j]$.

```
{
     $p := a[i];$ 
     $a[i] := a[j]; a[j] := p;$ 
}
```

3. Merge Sort

```
Algorithm MergeSort(low, high)  
// a[low : high] is a global array to be sorted.  
// Small(P) is true if there is only one element  
// to sort. In this case the list is already sorted.  
{  
    if (low < high) then // If there are more than one element  
    {  
        // Divide P into subproblems.  
        // Find where to split the set.  
        mid :=  $\lfloor (low + high)/2 \rfloor$ ;  
        // Solve the subproblems.  
        MergeSort(low, mid);  
        MergeSort(mid + 1, high);  
        // Combine the solutions.  
        Merge(low, mid, high);  
    }  
}
```

3. Merge Sort

```
Algorithm Merge(low, mid, high)
// a[low : high] is a global array containing two sorted
// subsets in a[low : mid] and in a[mid + 1 : high]. The goal
// is to merge these two sets into a single set residing
// in a[low : high]. b[ ] is an auxiliary global array.
{
    h := low; i := low; j := mid + 1;
    while ((h ≤ mid) and (j ≤ high)) do
    {
        if (a[h] ≤ a[j]) then
        {
            b[i] := a[h]; h := h + 1;
        }
        else
        {
            b[i] := a[j]; j := j + 1;
        }
        i := i + 1;
    }
    if (h > mid) then
        for k := j to high do
        {
            b[i] := a[k]; i := i + 1;
        }
    else
        for k := h to mid do
        {
            b[i] := a[k]; i := i + 1;
        }
    for k := low to high do a[k] := b[k];
}
```


4. Simple Union

Algorithm SimpleUnion(i, j)
{
 $p[i] := j$;
}

5. Simple Find

```
Algorithm SimpleFind( $i$ )  
{  
    while ( $p[i] \geq 0$ ) do  $i := p[i]$ ;  
    return  $i$ ;  
}
```

6. Weighted Union

Algorithm WeightedUnion(i, j)
// Union sets with roots i and j , $i \neq j$, using the
// weighting rule. $p[i] = -count[i]$ and $p[j] = -count[j]$.
{
 $temp := p[i] + p[j]$;
 if ($p[i] > p[j]$) **then**
 { // i has fewer nodes.
 $p[i] := j$; $p[j] := temp$;
 }
 else
 { // j has fewer or equal nodes.
 $p[j] := i$; $p[i] := temp$;
 }
}

7. Collapsing Find

Algorithm CollapsingFind(i)

```
// Find the root of the tree containing element  $i$ . Use the
// collapsing rule to collapse all nodes from  $i$  to the root.
{
     $r := i$ ;
    while ( $p[r] > 0$ ) do  $r := p[r]$ ; // Find the root.
    while ( $i \neq r$ ) do // Collapse nodes from  $i$  to root  $r$ .
    {
         $s := p[i]$ ;  $p[i] := r$ ;  $i := s$ ;
    }
    return  $r$ ;
}
```

Unit-3 Algorithms (5)

Dynamic Programming (5):

1. All Pairs Shortest Path
2. Optimal Binary Search Trees (OBST)
3. Travelling Salesperson Problem (TSP) **-No Algorithm**
4. Optimal Rod Cutting
 - Top Down Approach
 - Bottom Up Approach
5. 0/1 Knapsack Problem

1. All Pairs Shortest Path

```
0  Algorithm AllPaths(cost, A, n)
1  // cost[1 : n, 1 : n] is the cost adjacency matrix of a graph with
2  // n vertices; A[i, j] is the cost of a shortest path from vertex
3  // i to vertex j. cost[i, i] = 0.0, for  $1 \leq i \leq n$ .
4  {
5      for i := 1 to n do
6          for j := 1 to n do
7              A[i, j] := cost[i, j]; // Copy cost into A.
8          for k := 1 to n do
9              for i := 1 to n do
10                 for j := 1 to n do
11                     A[i, j] := min(A[i, j], A[i, k] + A[k, j]);
12 }
```

2. Optimal Binary Search Trees (OBST)

```
Algorithm OBST( $p, q, n$ )
// Given  $n$  distinct identifiers  $a_1 < a_2 < \dots < a_n$  and probabilities
//  $p[i]$ ,  $1 \leq i \leq n$ , and  $q[i]$ ,  $0 \leq i \leq n$ , this algorithm computes
// the cost  $c[i, j]$  of optimal binary search trees  $t_{ij}$  for identifiers
//  $a_{i+1}, \dots, a_j$ . It also computes  $r[i, j]$ , the root of  $t_{ij}$ .
//  $w[i, j]$  is the weight of  $t_{ij}$ .
{
    for  $i := 0$  to  $n - 1$  do
    {
        // Initialize.
         $w[i, i] := q[i]$ ;  $r[i, i] := 0$ ;  $c[i, i] := 0.0$ ;
        // Optimal trees with one node
         $w[i, i + 1] := q[i] + q[i + 1] + p[i + 1]$ ;
         $r[i, i + 1] := i + 1$ ;
         $c[i, i + 1] := q[i] + q[i + 1] + p[i + 1]$ ;
    }
     $w[n, n] := q[n]$ ;  $r[n, n] := 0$ ;  $c[n, n] := 0.0$ ;
    for  $m := 2$  to  $n$  do // Find optimal trees with  $m$  nodes.
        for  $i := 0$  to  $n - m$  do
        {
             $j := i + m$ ;
             $w[i, j] := w[i, j - 1] + p[j] + q[j]$ ;
            // Solve 5.12 using Knuth's result.
             $k := \text{Find}(c, r, i, j)$ ;
            // A value of  $l$  in the range  $r[i, j - 1] \leq l$ 
            //  $\leq r[i + 1, j]$  that minimizes  $c[i, l - 1] + c[l, j]$ ;
             $c[i, j] := w[i, j] + c[i, k - 1] + c[k, j]$ ;
             $r[i, j] := k$ ;
        }
    write ( $c[0, n]$ ,  $w[0, n]$ ,  $r[0, n]$ );
}
```

4. Optimal Rod Cutting - Top Down Approach

Algorithm:

Recursive:

CUT-ROD(p, n)

if $n == 0$:

return 0

$q = -\infty$

for $i=1$ to n

$q = \max(q, p[i] + \text{CUT-ROD}(p, n-i))$

return q

Recursive calls:

CUT-ROD(p, 4)

4. Optimal Rod Cutting - Bottom Up Approach

Iterative

CUT-ROD(p, n)

Let $r[0 \dots n]$ be an array

for $i = 1$ to n

$q = -\infty$

 for $j = 1$ to i

$q = \max(q, p[j] + r[i-j])$

$r[i] = q$

return $r[n]$

5. 0/1 Knapsack Problem

```
1  Algorithm DKP( $p, w, n, m$ )
2  {
3       $S^0 := \{(0, 0)\}$ ;
4      for  $i := 1$  to  $n - 1$  do
5      {
6           $S_1^{i-1} := \{(P, W) | (P - p_i, W - w_i) \in S^{i-1} \text{ and } W \leq m\}$ ;
7           $S^i := \text{MergePurge}(S^{i-1}, S_1^{i-1})$ ;
8      }
9       $(PX, WX) := \text{last pair in } S^{n-1}$ ;
10      $(PY, WY) := (P' + p_n, W' + w_n)$  where  $W'$  is the largest  $W$  in
11         any pair in  $S^{n-1}$  such that  $W + w_n \leq m$ ;
12     // Trace back for  $x_n, x_{n-1}, \dots, x_1$ .
13     if  $(PX > PY)$  then  $x_n := 0$ ;
14     else  $x_n := 1$ ;
15     TraceBackFor( $x_{n-1}, \dots, x_1$ );
16 }
```

Unit-4 Algorithms (9)

Greedy Method (6):

1. Job Sequencing With DeadLines
2. Fractional Knapsack Problem
3. Minimum Spanning Trees
 - Prim's Algorithm
 - Kruskal's Algorithm
4. Single Source Shortest Path
5. Activity Selection Problem

Backtracking (3):

6. N-Queens Problem
7. Hamiltonian Cycles
8. Sum Of Subsets

1. Job Sequencing With DeadLines

```
1  Algorithm JS( $d, j, n$ )
2  //  $d[i] \geq 1, 1 \leq i \leq n$  are the deadlines,  $n \geq 1$ . The jobs
3  // are ordered such that  $p[1] \geq p[2] \geq \dots \geq p[n]$ .  $J[i]$ 
4  // is the  $i$ th job in the optimal solution,  $1 \leq i \leq k$ .
5  // Also, at termination  $d[J[i]] \leq d[J[i+1]], 1 \leq i < k$ .
6  {
7       $d[0] := J[0] := 0$ ; // Initialize.
8       $J[1] := 1$ ; // Include job 1.
9       $k := 1$ ;
10     for  $i := 2$  to  $n$  do
11     {
12         // Consider jobs in nonincreasing order of  $p[i]$ . Find
13         // position for  $i$  and check feasibility of insertion.
14          $r := k$ ;
15         while  $((d[J[r]] > d[i])$  and  $(d[J[r]] \neq r))$  do  $r := r - 1$ ;
16         if  $((d[J[r]] \leq d[i])$  and  $(d[i] > r))$  then
17         {
18             // Insert  $i$  into  $J[ ]$ .
19             for  $q := k$  to  $(r + 1)$  step  $-1$  do  $J[q + 1] := J[q]$ ;
20              $J[r + 1] := i$ ;  $k := k + 1$ ;
21         }
22     }
23     return  $k$ ;
24 }
```

2. Fractional Knapsack Problem

```
1  Algorithm GreedyKnapsack( $m, n$ )
2  //  $p[1 : n]$  and  $w[1 : n]$  contain the profits and weights respectively
3  // of the  $n$  objects ordered such that  $p[i]/w[i] \geq p[i+1]/w[i+1]$ .
4  //  $m$  is the knapsack size and  $x[1 : n]$  is the solution vector.
5  {
6      for  $i := 1$  to  $n$  do  $x[i] := 0.0$ ; // Initialize  $x$ .
7       $U := m$ ;
8      for  $i := 1$  to  $n$  do
9      {
10         if ( $w[i] > U$ ) then break;
11          $x[i] := 1.0$ ;  $U := U - w[i]$ ;
12     }
13     if ( $i \leq n$ ) then  $x[i] := U/w[i]$ ;
14 }
```

3. Minimum Spanning Trees - Prim's Algorithm

```
Algorithm Prim( $E, cost, n, t$ )  
//  $E$  is the set of edges in  $G$ .  $cost[1 : n, 1 : n]$  is the cost  
// adjacency matrix of an  $n$  vertex graph such that  $cost[i, j]$  is  
// either a positive real number or  $\infty$  if no edge  $(i, j)$  exists.  
// A minimum spanning tree is computed and stored as a set of  
// edges in the array  $t[1 : n - 1, 1 : 2]$ .  $(t[i, 1], t[i, 2])$  is an edge in  
// the minimum-cost spanning tree. The final cost is returned.  
{  
    Let  $(k, l)$  be an edge of minimum cost in  $E$ ;  
     $mincost := cost[k, l]$ ;  
     $t[1, 1] := k$ ;  $t[1, 2] := l$ ;  
    for  $i := 1$  to  $n$  do // Initialize near.  
        if  $(cost[i, l] < cost[i, k])$  then  $near[i] := l$ ;  
        else  $near[i] := k$ ;  
     $near[k] := near[l] := 0$ ;  
    for  $i := 2$  to  $n - 1$  do  
    { // Find  $n - 2$  additional edges for  $t$ .  
        Let  $j$  be an index such that  $near[j] \neq 0$  and  
         $cost[j, near[j]]$  is minimum;  
         $t[i, 1] := j$ ;  $t[i, 2] := near[j]$ ;  
         $mincost := mincost + cost[j, near[j]]$ ;  
         $near[j] := 0$ ;  
        for  $k := 1$  to  $n$  do // Update near[ ].  
            if  $((near[k] \neq 0) \text{ and } (cost[k, near[k]] > cost[k, j]))$   
            then  $near[k] := j$ ;  
    }  
} return  $mincost$ ;
```

3. Minimum Spanning Trees - Kruskal's Algorithm

Algorithm $\text{Kruskal}(E, \text{cost}, n, t)$

// E is the set of edges in G . G has n vertices. $\text{cost}[u, v]$ is the
// cost of edge (u, v) . t is the set of edges in the minimum-cost
// spanning tree. The final cost is returned.

```
{
    Construct a heap out of the edge costs using Heapify;
    for  $i := 1$  to  $n$  do  $\text{parent}[i] := -1$ ;
    // Each vertex is in a different set.
     $i := 0$ ;  $\text{mincost} := 0.0$ ;
    while  $((i < n - 1)$  and (heap not empty)) do
    {
        Delete a minimum cost edge  $(u, v)$  from the heap
        and reheapify using Adjust;
         $j := \text{Find}(u)$ ;  $k := \text{Find}(v)$ ;
        if  $(j \neq k)$  then
        {
             $i := i + 1$ ;
             $t[i, 1] := u$ ;  $t[i, 2] := v$ ;
             $\text{mincost} := \text{mincost} + \text{cost}[u, v]$ ;
            Union( $j, k$ );
        }
    }
    if  $(i \neq n - 1)$  then write ("No spanning tree");
    else return  $\text{mincost}$ ;
}
```

4. Single Source Shortest Path

Algorithm ShortestPaths($v, cost, dist, n$)
// $dist[j]$, $1 \leq j \leq n$, is set to the length of the shortest
// path from vertex v to vertex j in a digraph G with n
// vertices. $dist[v]$ is set to zero. G is represented by its
// cost adjacency matrix $cost[1 : n, 1 : n]$.
{
 for $i := 1$ to n do
 { // Initialize S .
 $S[i] := \text{false}$; $dist[i] := cost[v, i]$;
 }
 $S[v] := \text{true}$; $dist[v] := 0.0$; // Put v in S .
 for $num := 2$ to $n - 1$ do
 {
 // Determine $n - 1$ paths from v .
 Choose u from among those vertices not
 in S such that $dist[u]$ is minimum;
 $S[u] := \text{true}$; // Put u in S .
 for (each w adjacent to u with $S[w] = \text{false}$) do
 // Update distances.
 if ($dist[w] > dist[u] + cost[u, w]$) then
 $dist[w] := dist[u] + cost[u, w]$;
 }
}

5. Activity Selection Problem

Activity-Selection(Activity, start, finish)

**Sort Activities by finish times stored
in finish**

Selected = {Activity[1]}

n = Activity.length

j = 1

for i = 2 to n:

if start[i] $\geq$ finish[j]:

Selected = Selected U Activity[i]}

j = i

return Selected

6. N-Queens Problem

```
1  Algorithm NQueens( $k, n$ )
2  // Using backtracking, this procedure prints all
3  // possible placements of  $n$  queens on an  $n \times n$ 
4  // chessboard so that they are nonattacking.
5  {
6      for  $i := 1$  to  $n$  do
7      {
8          if Place( $k, i$ ) then
9          {
10              $x[k] := i$ ;
11             if ( $k = n$ ) then write ( $x[1 : n]$ );
12             else NQueens( $k + 1, n$ );
13         }
14     }
15 }
```

7. Hamiltonian Cycles

```
1  Algorithm Hamiltonian( $k$ )
2  // This algorithm uses the recursive formulation of
3  // backtracking to find all the Hamiltonian cycles
4  // of a graph. The graph is stored as an adjacency
5  // matrix  $G[1 : n, 1 : n]$ . All cycles begin at node 1.
6  {
7      repeat
8      { // Generate values for  $x[k]$ .
9          NextValue( $k$ ); // Assign a legal next value to  $x[k]$ .
10         if ( $x[k] = 0$ ) then return;
11         if ( $k = n$ ) then write ( $x[1 : n]$ );
12         else Hamiltonian( $k + 1$ );
13     } until (false);
14 }
```

8. Sum Of Subsets

```
1  Algorithm SumOfSub( $s, k, r$ )
2  // Find all subsets of  $w[1 : n]$  that sum to  $m$ . The values of  $x[j]$ ,
3  //  $1 \leq j < k$ , have already been determined.  $s = \sum_{j=1}^{k-1} w[j] * x[j]$ 
4  // and  $r = \sum_{j=k}^n w[j]$ . The  $w[j]$ 's are in nondecreasing order.
5  // It is assumed that  $w[1] \leq m$  and  $\sum_{i=1}^n w[i] \geq m$ .
6  {
7      // Generate left child. Note:  $s + w[k] \leq m$  since  $B_{k-1}$  is true.
8       $x[k] := 1$ ;
9      if ( $s + w[k] = m$ ) then write ( $x[1 : k]$ ); // Subset found
10     // There is no recursive call here as  $w[j] > 0$ ,  $1 \leq j \leq n$ .
11     else if ( $s + w[k] + w[k + 1] \leq m$ )
12         then SumOfSub( $s + w[k], k + 1, r - w[k]$ );
13     // Generate right child and evaluate  $B_k$ .
14     if (( $s + r - w[k] \geq m$ ) and ( $s + w[k + 1] \leq m$ )) then
15     {
16          $x[k] := 0$ ;
17         SumOfSub( $s, k + 1, r - w[k]$ );
18     }
19 }
```

Unit-5 Algorithms (3)

Branch & Bound (3):

1. Travelling Sales Person With LCBB **-No Algorithm**
2. 0/1 Knapsack With LCBB
 - Upper bound
 - Lower bound
3. 0/1 Knapsack With FIFOBB
 - Upper bound
 - Lower bound

0/1 Knapsack - Upper Bound

```
1  Algorithm UBound(cp, cw, k, m)
2  // cp, cw, k, and m have the same meanings as in
3  // Algorithm 7.11. w[i] and p[i] are respectively
4  // the weight and profit of the ith object.
5  {
6      b := cp; c := cw;
7      for i := k + 1 to n do
8          {
9              if (c + w[i] ≤ m) then
10                 {
11                     c := c + w[i]; b := b − p[i];
12                 }
13             }
14     return b;
15 }
```

0/1 Knapsack - Lower Bound

```
1  Algorithm Bound(cp, cw, k)
2  // cp is the current profit total, cw is the current
3  // weight total; k is the index of the last removed
4  // item; and m is the knapsack size.
5  {
6      b := cp; c := cw;
7      for i := k + 1 to n do
8          {
9              c := c + w[i];
9              if (c < m) then b := b + p[i];
10             else return b + (1 - (c - m)/w[i]) * p[i];
11         }
12     return b;
13 }
```